

GRADUATION PROJECT DOCUMENTATION

RASED PROJECT

Comprehensive Project Summary, Implemented Functions, & Design Alternatives

Project Type:	ASP.NET Core Web API (Clean Architecture)
Date:	July 9, 2026
Target OS:	Cross-Platform (Windows / Linux)
Database:	Entity Framework Core / SQL Server

1. Project Overview & Architecture

The **Rased Project** is a backend solution designed to manage and optimize passenger and driver interactions, facilitate complaint logging with OCR license plate recognition capabilities, and support real-time locations and announcements/advertisements.

The backend architecture is structured around **Clean Architecture (Onion Architecture)** principles, promoting loose coupling and ease of maintainability:

Layer	Responsibilities	Primary Directories & Components
Rased.Core CORE	Domain entities, contracts, DTO configurations, and enumeration sets. Independent of database technologies.	Entities/ , Identity/ , DTO/ , Enums/ , ServiceContracts/
Rased.Infrastructure INFRASTRUCTURE	Database contexts, migrations, repositories, and concrete business logic services.	DbContext/ , Migrations/ , Services/
Rased Project (UI) PRESENTATION	Web API controllers, Swagger generation, routing pipelines, authentication middlewares, and CORS setup.	Controllers/ , Program.cs , appsettings.json

2. Active Services & Implemented Functions

Below is the detailed registry of the services, contracts, and core methods implemented in the project codebase:

A. Authentication & Profile Management

Contract: `IAccountService.cs` | **Implementation:** `AccountService.cs`

- `PostRegister(RegisterDTO)` : Handles credentials validation and uses ASP.NET Identity `UserManager` to register users. Creates linked `UserProfile` entries, assigns roles (`Driver` / `Passenger`), and dispatches registration email OTP codes.
- `PostLogin(LoginDTO)` : Authenticates credentials, pulls roles, and issues a JWT token via `IJWTService` containing embedded roles.
- `GetProfile(Guid userId)` : Gathers user profile information (including `PlateNumber` and `SSN` for drivers) by querying the database `UserProfile` table.
- `UpdateProfile(Guid userId, UpdateProfileDto)` : Allows users to update profile names, license plates, and user phone numbers.
- `ChangePassword(Guid userId, ChangePasswordDto)` : Executes password changes with secure validations.
- `ForgotPasswordSendOtp(SendOtpDto)` & `ResetPasswordWithOtp(ResetPasswordWithOtpDto)` : Coordinates out-of-band password recovery workflows.

B. OTP Generation & Verification

Contract: `IOTPService.cs` | **Implementation:** `OTPService.cs`

- `SendOtpAsync(SendOtpDto)` : Dynamically generates a secure 6-digit OTP code, saves it inside an `IMemoryCache` with a sliding expiration of 5 minutes, and triggers email delivery.
- `VerifyOtpAsync(VerifyOtpDto)` / `ValidateOtpAsync(...)` : Validates codes. Incorporates safety controls (up to 5 failed verification attempts before cache eviction).

C. Complaint Logging & Dashboard Management

Contract: `IComplaintService.cs` | **Implementation:** `ComplaintService.cs`

- `CreateAsync(CreateComplaintDto, ClaimsPrincipal)` : Inserts user complaints with description, GPS coordinates, human-readable address names, and photo/video attachments. Generates a unique tracking Serial Number.
- `GetAllAsync(string status, string search)` : Pulls complaints for the dashboard interface. Supports robust filters and search capabilities:

Advanced Search & Filtering

Searches by Serial Number, Description, Submitting User Full Name, or the OCR-read License Plate (`AIGeneratedText`).

- `UpdateStatusAsync(Guid id, UpdateComplaintStatusDto)` : Updates complaint states (Pending, Resolved, Rejected). It synchronizes the database text field `complaintStatus` and code field `Status` (0 = Pending, 1 = Resolved, 2 = Rejected) to keep mobile and web views consistent.
- `GetStatsAsync()` : Computes statistical metrics (active daily complaints count, state breakdown, and overall user/driver counts).

D. Real-Time Location Sharing & Hotspot Analysis

Implementation: `RideMatchingService.cs`

- `UpdateUserLocationAsync(Guid, UpdateLocationDto)` : Stores or updates live geographic positions of passengers and drivers.
- `GetActivePassengersAsync(double driverLat, double driverLng, double radiusInKm)` : Calculates passenger positions inside the driver's geographic radius using the **Haversine formula**.
- `GetHotspotsAsync(double radiusInMeters)` : Aggregates passenger count trends around defined gathering spots, guiding drivers to high-demand locations.

3. Retired Alternatives & Architectural Shifts

During the lifecycle of the project, several components and architectures were refactored or completely replaced. Below is a detailed look at those choices:

Previous / Alternative Design	Active Implemented Solution	Reasoning & Benefits of the Decision
SMS-Based Registration & OTP <code>ISmsSender.cs</code> / <code>ConsoleSmsSender.cs</code> (Deleted)	Email SMTP Verification <code>IEmailSender.cs</code> / <code>SmtplibEmailSender.cs</code>	Replaced SMS configurations with SMTP email verification. This removed dependency on third-party SMS gateway providers, significantly lowered transactional overhead costs, and avoided sandbox carrier limitations during testing.
Complaint Categories Module <code>Category.cs</code> link, <code>GetCategoriesAsync()</code>	Unified Direct Complaints Category link removed from <code>Complaint.cs</code>	Decoupling complaints from strict categorization simplified the report logging pipeline. Users can report incidents directly using text descriptions, images/videos, and locations, rather than navigate rigid category selectors.
Service-Driven Ad Operations <code>IAdsService.cs</code> / <code>AdsService.cs</code> (Registered but Unused)	Direct Context Querying Direct DB queries in <code>AdsController.cs</code>	For simple operations like CRUD on Ads, bypassing the service layer avoided boilerplate code. The controller directly queries <code>ApplicationDbContext</code> .
Unified Monolithic Identity Tables	Split User & Profile Tables <code>ApplicationUser</code> &	Separates ASP.NET Core Identity tables from metadata

Previous / Alternative Design	Active Implemented Solution	Reasoning & Benefits of the Decision
	<code>UserProfile</code> (1-to-1 relationship)	properties (like <code>SSN</code> and <code>PlateNumber</code>). Keeps authentication schemas lightweight and optimizes query performance.

Note on Legacy Registrations

Although `IAdsService` / `AdsService` is fully registered in `Program.cs` , the API controller utilizes direct Entity Framework queries as the active pipeline. Similarly, the `Category` entity remains in the codebase as a legacy artifact but is disconnected from active complaint flows.

4. Database Schema Structure

The EF Core model maps directly to the following entity schemas:

- **ApplicationUser** : Extends Identity. Stores security flags, password hashes, and the activation boolean (`IsActive`).
- **UserProfile** : Connected to User via a 1-to-1 relationship. Stores `FullName` , `SSN` , `PlateNumber` , and `ProfilePictureUrl` .
- **Complaint** : Stores reports, linking to a user. Includes image/video file links, coordinate tracking (`Lat` / `Lng`), geolocations, OCR text (`AIGeneratedText`), and status parameters (`complaintStatus` and `Status`).
- **Ad** : Holds system-wide announcements and advertisements.
- **UserLiveLocation** : Keeps track of live passenger/driver status, timestamps, and coordinates.
- **GatheringSpot** : Represents hotspots database containing predefined gathering regions.